TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 4
Notificaciones

Locales y push · `expo-notifications`

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Expo SDK 57 · React Native 0.86 · React 19.2

UNIDAD 4

Qué vamos a ver

Objetivos

Programar, cancelar y reaccionar a notificaciones locales.

Entender el recorrido completo de una notificación push.

Obtener y administrar un token de dispositivo.

Diagnosticar por qué una push no llegó.

Partes

Qué es una notificación

Notificaciones locales

Notificaciones push

Buenas prácticas y diagnóstico

> Las **locales** se prueban en Expo Go y no necesitan servidor: es donde vamos a pasar la mayor parte de la clase.

UNIDAD 4 · PARTE 1

Qué es una notificación

El único canal que tenés cuando la app está cerrada.

UNIDAD 4 · PARTE 1 · QUÉ ES UNA NOTIFICACIÓN

Lo que la vuelve distinta



La app está cerrada. El sistema dibuja esto, no vos.

No es una pantalla de tu app

La dibuja el sistema operativo. Vos aportás título, cuerpo y datos; el aspecto y el momento los decide él.

Es un permiso, y de los caros

El usuario lo concede una vez y lo revoca a la primera molestia. Una app que abusa pierde el canal para siempre.

Es el único canal con la app cerrada

Sin notificaciones, tu app solo existe mientras alguien la está mirando.

UNIDAD 4 · PARTE 1 · QUÉ ES UNA NOTIFICACIÓN

Locales y push

LOCAL ARRANCAMOS POR ACÁ

La programa **la propia app**, en el dispositivo.

No necesita internet ni servidor.

El sistema la guarda y la dispara aunque la app esté cerrada.

Recordatorios, temporizadores, alarmas, vencimientos.

Funciona en Expo Go.

PUSH

La manda **un servidor**, en cualquier momento.

Necesita conexión, un token por dispositivo y credenciales.

Pasa por Google (FCM) o Apple (APNs).

Mensajes nuevos, cambios de estado de un pedido.

Necesita development build.

> Existe un tercer tipo, la **push silenciosa**: no muestra nada, solo despierta la app para que corra un poco de JavaScript en segundo

> plano (sincronizar datos, por ejemplo). Sale del alcance de la materia.

UNIDAD 4 · PARTE 1 · QUÉ ES UNA NOTIFICACIÓN

Quién decide qué se ve

App en primer plano

El usuario está mirando tu app. **Decidís vos**, con `setNotificationHandler`.

Por defecto no se muestra nada: si no configurás el handler, parece que la notificación se perdió.

App en segundo plano

Minimizada. **Decide el sistema**: muestra el banner y lo deja en el centro de notificaciones.

App cerrada

Terminada por el usuario o por el sistema. También decide el sistema; tu código ni se ejecuta.

> El «no me llega la notificación» número uno de la cursada: **la app estaba abierta y no había handler**. Probar siempre con la app minimizada antes de salir a buscar el problema en otro lado.

UNIDAD 4 · PARTE 2

Notificaciones locales

Sin servidor, sin internet, sin credenciales.

UNIDAD 4 · PARTE 2 · NOTIFICACIONES LOCALES

Instalar y configurar

```
{
  "expo": {
    "plugins": [
      ["expo-notifications", {
        "icon": "./assets/notificacion.png",
        "color": "#0B3A5D",
        "sounds": ["./assets/aviso.wav"]
      }]
    ]
  }
}
```

ICON Y COLOR

Solo Android. Silueta blanca sobre transparente: el sistema la recorta.

SOUNDS

Se empaquetan en el binario y después se referencian por nombre.

> Es un config plugin: tocarlo **obliga a compilar de nuevo**.

UNIDAD 4 · PARTE 2 · NOTIFICACIONES LOCALES

El handler: qué pasa con la app abierta

```
import * as Notifications from 'expo-notifications';

// Se declara UNA vez, fuera del componente, al
cargar el módulo.
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowBanner: true,    // el cartel que baja
arriba
    shouldShowList: true,      // queda en el centro
de notificaciones
    shouldPlaySound: true,
    shouldSetBadge: false,     // el globito rojo
sobre el ícono
  }),
});
```

> `handleNotification` recibe la notificación, así que podés decidir por contenido: si el usuario ya está en la pantalla del chat, devolvé `shouldShowBanner: false`.

UNIDAD 4 · PARTE 2 · NOTIFICACIONES LOCALES

Permisos y canales

```
async function pedirPermiso() {
  if (Platform.OS === 'android') {        // el canal
va ANTES del permiso
    await
Notifications.setNotificationChannelAsync('default',
{
      name: 'Avisos generales',
      importance:
Notifications.AndroidImportance.MAX,
      vibrationPattern: [0, 250, 250, 250],
    });
  }

  const { status } = await
Notifications.getPermissionsAsync();
  if (status === 'granted') return true;

  const pedido = await
Notifications.requestPermissionsAsync();
  return pedido.status === 'granted';
}
```

> Un **canal** de Android es una categoría que el usuario puede silenciar por separado —«promociones» sí, «mensajes» no—. Sin al menos un canal creado, en Android 13 o superior el cartel de permiso **no aparece**.

UNIDAD 4 · PARTE 2 · NOTIFICACIONES LOCALES

Programar

AHORA MISMO

```
await Notifications.scheduleNotificationAsync({
  content: {
    title: 'Listo',
    body: 'Se guardó tu entrega.',
    data: { ruta: '/entregas/42' },
  },
  trigger: null,
});
```

EN UN RATO, Y REPITIENDO

```
trigger: {
  type: Notifications.SchedulableTriggerInputTypes
          .TIME_INTERVAL,
  seconds: 60 * 20,
  repeats: true,
}
```

EN UNA FECHA EXACTA

```
trigger: {
  type: Notifications.SchedulableTriggerInputTypes
          .DATE,
  date: new Date('2026-11-14T09:00:00'),
}
```

Hay más tipos —diario, semanal, coincidencia de calendario en iOS—. Están en `SchedulableTriggerInputTypes`.

> El `trigger` ahora **lleva** **`type`**. Si ves `trigger: { seconds: 60 }` suelto, es la forma vieja DEPRECADA.

UNIDAD 4 · PARTE 2 · NOTIFICACIONES LOCALES

Administrar lo programado

```
// Guardá el identificador: es lo único que te deja cancelar una sola.
const id = await Notifications.scheduleNotificationAsync({ /* … */ });

await Notifications.cancelScheduledNotificationAsync(id);
await Notifications.cancelAllScheduledNotificationsAsync();

const pendientes = await Notifications.getAllScheduledNotificationsAsync();
console.log(pendientes.length);

// Distinto: esto limpia lo que YA se mostró y está en el centro de notificaciones.
await Notifications.dismissAllNotificationsAsync();
```

> **Cancelar ≠ descartar.** `cancel…` borra algo que todavía no se disparó; `dismiss…` saca de la bandeja algo que ya apareció. Y reprogramar sin cancelar duplica: si el usuario edita el recordatorio tres veces, recibe tres notificaciones.

UNIDAD 4 · PARTE 2 · NOTIFICACIONES LOCALES

Reaccionar al toque

```
const router = useRouter();

useEffect(() => {
  // Llegó mientras la app estaba abierta.
  const recibida =
Notifications.addNotificationReceivedListener((n) =>
{
    console.log('llegó:', n.request.content.title);
  });

  // El usuario la apretó: la app puede estar
abierta, minimizada o cerrada.
  const tocada =
Notifications.addNotificationResponseReceivedListener
((r) => {
    const { ruta } =
r.notification.request.content.data;
    if (ruta) router.push(ruta);
  });

  return () => { recibida.remove(); tocada.remove();
};
}, []);
```

> Por eso el campo `data` lleva la ruta y no el texto: es lo que convierte la notificación en un atajo a una pantalla concreta. Con Expo Router, `router.push(ruta)` y listo.

UNIDAD 4 · PARTE 3

Notificaciones push

Del servidor al teléfono, y todo lo que hay en el medio.

UNIDAD 4 · PARTE 3 · NOTIFICACIONES PUSH

El recorrido completo

1 · La app

Pide permiso y genera un **token**: `ExponentPushToken[…]` . Se lo manda a tu servidor.

2 · Tu servidor

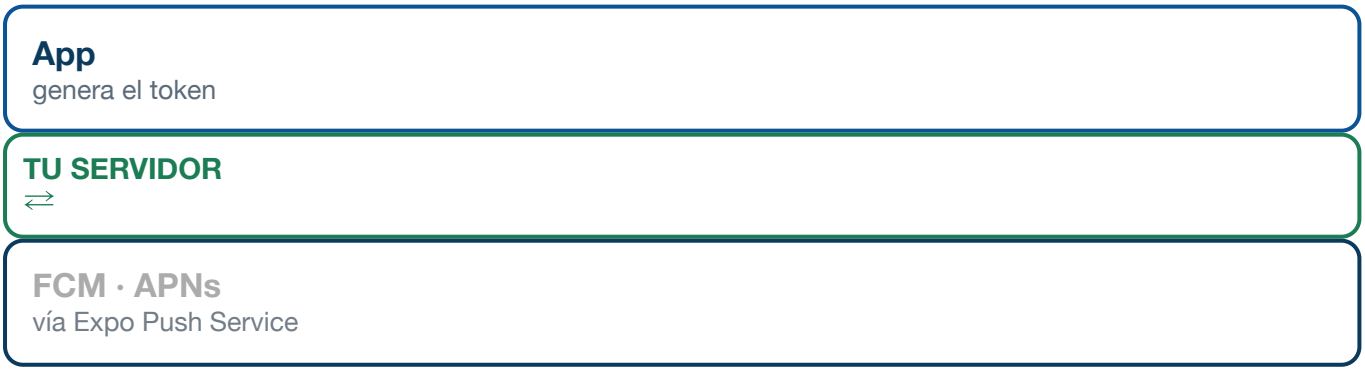
Guarda el token asociado al usuario. Cuando pasa algo, arma el mensaje.

3 · Expo Push Service

Recibe el mensaje, mira el token y decide si va por Google o por Apple.

4 · FCM / APNs

El servicio de Google o Apple lo entrega al dispositivo. Es el único que puede.



UNIDAD 4 · PARTE 3 · NOTIFICACIONES PUSH

Conseguir el token

```
import * as Notifications from 'expo-notifications';
import * as Device from 'expo-device';
import Constants from 'expo-constants';

export async function registrarParaPush() {
  if (!Device.isDevice) return null;
// 1 · ¿dispositivo real?
  if (!(await pedirPermiso())) return null;
// 2 · permiso + canal

  const projectId =
Constants.expoConfig?.extra?.eas?.projectId;   // 3 ·
proyecto
  if (!projectId) throw new Error('Falta el projectId
de EAS');

  const { data: token } =
    await Notifications.getExpoPushTokenAsync({
projectId });      // 4 · token

  await fetch('https://mi-api/tokens', {
// 5 · al servidor
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ token }),
  });

  return token;   //
```

```
ExponentPushToken[xxxxxxxxxxxxxxxxxxxxxx]
}
```

UNIDAD 4 · PARTE 3 · NOTIFICACIONES PUSH

Qué hace falta antes de que esto ande

Development build

Expo Go no sirve para push en Android desde el SDK 53.

```
eas build --profile development
```

Credenciales

Una clave de APNs para iOS y las de FCM para Android.

`eas credentials` las genera y las guarda.

Dónde probar

Dispositivo físico, emulador de Android con Google Play, o simulador de iOS con Xcode 14 o superior.

UNIDAD 4 · PARTE 3 · NOTIFICACIONES PUSH

Mandar una push

PARA PROBAR — CURL

```
curl -X POST https://exp.host/--/api/v2/push/send
\
  -H "Content-Type: application/json" \
  -d '{
    "to": "ExponentPushToken[xxxxxxxx]",
    "title": "Se corrigió tu entrega",
    "body": "Entrá para ver la devolución.",
    "data": { "ruta": "/entregas/42" }
  }'
```

CÓDIGO DE SERVIDOR — EXPO-SERVER-SDK

```
import { Expo } from 'expo-server-sdk';
const expo = new Expo();

const mensajes = tokens
  .filter(t => Expo.isExpoPushToken(t))
  .map(to => ({ to, title: 'Novedad', body: '…'
}));

const tickets = [];
for (const lote of
expo.chunkPushNotifications(mensajes)) {
  tickets.push(...await
expo.sendPushNotificationsAsync(lote));
}
```

> El SDK de servidor no es un lujo: parte en lotes de 100, limita las conexiones en paralelo y valida el formato del token. Hacerlo a mano con `fetch` funciona hasta que tenés doscientos usuarios.

UNIDAD 4 · PARTE 3 · NOTIFICACIONES PUSH

Tickets y recibos: no son lo mismo

Ticket — inmediato

Respuesta de Expo al recibir tu mensaje. `status: "ok"` significa **«lo tomé»**, no «llegó».

Trae un `id` para consultar después.

Recibo — a los ~15 minutos

Dice qué contestaron Google o Apple. Es el único lugar donde te enterás de que un token murió.

```
await expo.getPushNotificationReceiptsAsync(ids);
```

> Si tu servidor no lee los recibos, va a seguir mandando mensajes a teléfonos que desinstalaron la app. Con el tiempo Expo empieza a limitarte por tasa de error.

UNIDAD 4 · PARTE 3 · NOTIFICACIONES PUSH

Los errores que vas a ver

DeviceNotRegistered

Desinstaló la app o revocó el permiso. **Borrá ese token de tu base de datos**, no reintentes.

MessageTooBig

El mensaje completo pasa de 4 KB. Mandá un identificador en `data`, no el contenido entero.

MessageRateExceeded

Demasiados mensajes al mismo dispositivo. Reintento con espera creciente.

`MismatchSenderId` aparece en Android cuando las credenciales de FCM no corresponden al proyecto. Se arregla con `eas credentials`, no en el código.

Para probar rápido sin servidor: la **Expo Push Notifications Tool** en `expo.dev/notifications`. Pegás el token y mandás.

UNIDAD 4 · PARTE 3 · NOTIFICACIONES PUSH

Dónde vive cada cosa

EN LA APP

El token propio, y nada más.

Se regenera cuando hace falta: no hay que persistirlo con cuidado.

EN EL SERVIDOR

La tabla de tokens, asociada a usuarios.

La lógica de a quién y cuándo notificar.

Las credenciales de FCM y APNs.

> Un token no es un secreto grave, pero **la lista completa sí lo es**: con ella cualquiera le manda notificaciones a todos tus usuarios. Nunca la expongas en un endpoint público ni la metas en el repositorio.

UNIDAD 4 · PARTE 4

Buenas prácticas y diagnóstico

Cómo no perder el permiso, y qué mirar cuando no llega.

UNIDAD 4 · PARTE 4 · BUENAS PRÁCTICAS Y DIAGNÓSTICO

Cuándo notificar

Sí

Algo que el usuario pidió: un recordatorio que él programó.

Algo que lo involucra: le respondieron, le asignaron una tarea.

Algo con vencimiento real.

No

«Hace 3 días que no entrás».

Promociones sin que las haya pedido.

Cinco avisos separados de lo mismo en vez de uno agrupado.

De madrugada, porque tu servidor corre en otro huso horario.

> Dos reglas que valen para el TP: **pedí el permiso recién cuando el usuario activa algo que lo necesita**, y **dale una pantalla de ajustes** para elegir qué avisos quiere. Es más barato que perder el permiso entero.

UNIDAD 4 · PARTE 4 · BUENAS PRÁCTICAS Y DIAGNÓSTICO

No me llega. ¿Qué miro?

Primero, lo tonto

¿La app está en primer plano y falta el handler?

¿El permiso está concedido? ¿ `canAskAgain` quedó en `false` ?

¿Estás en Expo Go queriendo probar push?

¿El dispositivo está en «no molestar»?

Después, la cadena

¿El token se generó? Imprimilo.

¿La herramienta de Expo entrega a ese token?

¿Qué dice el **recibo**, no el ticket?

¿ `eas credentials` muestra clave de APNs y de FCM?

> Diagnosticá **en ese orden**. La mayoría de los casos se resuelven en la columna de la izquierda, y la de la derecha lleva media hora.

UNIDAD 4

Documentación

- docs.expo.dev/versions/latest/sdk/notifications
- docs.expo.dev/push-notifications/overview
- docs.expo.dev/push-notifications/push-notifications-setup
- docs.expo.dev/push-notifications/sending-notifications

Speaker notes